

## 第 3 章 需求分析

需求問題是造成軟體工程項目失敗的主要原因，能否開發出高品質的軟體，很大程度上取決於對要解決的問題的認識以及如何準確地表達出用戶的需求。通過需求分析使得分析者深刻地理解和認識系統，並將其完全、準確地表達，其結果不僅起到溝通（用戶和開發者）作用，還是後續工作的依據。本章介紹需求分析的一些基本概念，分別對需求獲取技術、需求規格說明書、如何進行需求分析以及需求分析方法進行討論，重點討論結構化的需求分析方法。

### 3.1 需求分析的概念和任務

#### 3.1.1 需求的概念

什麼是需求？到目前為止還沒有公認的定義。對使用者來講需求是對軟體產品的解釋，是使用者對目標軟體系統在功能、行為、性能、設計和約束等方面的期望；而開發人員所講的需求對用戶來說又像是詳細設計。比較權威的定義是 IEEE 軟體工程標準詞彙表中的需求定義：

- （1）使用者解決問題或達到目標所需的條件或權能（**Capability**）。
- （2）系統或系統部件要滿足合同、標準、規範或其他正式規定文檔所需具有的條件或權能。
- （3）一種反映上面（1）或（2）所描述的條件或權能的文檔說明。

由定義可知，需求一方面反映了系統的外部行為，另一方面反映了系統的內部特性，反映的方式是需求文檔。用規範的格式表達出來的文檔說明稱為需求規格說明書，或者簡稱為“需求說明”。

#### 3.1.2 需求的層次

需求可分解為 4 個層次：業務需求、用戶需求、功能需求和非功能需求。

（1）業務需求（**Business Requirement**）：業務需求是反映組織機構或客戶對軟體高層次的目標要求。這項需求是用戶高層領導機構決定的，它確定了系統的目標、規模和範圍。業務需求是需求分析階段制定需求調研計畫、確定使用者核心需求和軟體功能需求的依據，應在進行需求分析之前確定，通常在項目定義與範圍文檔中予以說明。

（2）用戶需求（**User Requirement**）：使用者需求是使用者使用該軟體要完成的任務。要弄清這部分需求，應該充分調研具體的業務部門，詳細瞭解最終用戶的工作過程、所涉及的資訊、當前系統的工作情況、與其他系統的介面等。用戶需求是最重要的需求，也是最容易出現問題的部分。

（3）功能需求（**Functional Requirement**）：功能需求定義了軟體必須實現的功能。由於使用者是從完成任務的角度對軟體提出需求的，通常是凌亂的、非系統化的、冗余的，開發人員無法據此



編寫程式。分析人員必須在充分理解用戶需求的基礎上，將用戶需求整理成滿足特定業務需求的軟體功能需求。

(4) 非功能需求：非功能需求是對功能需求的補充。可以分為兩類：一類是使用者關心的一些重要屬性，如有效性、效率、靈活性、完整性、互通性、可靠性、健壯性、可用性；另一類是對開發者來說很重要的品質屬性，如可維護性、可攜性、可複用性、可測試性。

軟體需求各組成部分之間的關係如圖 3-1 所示。

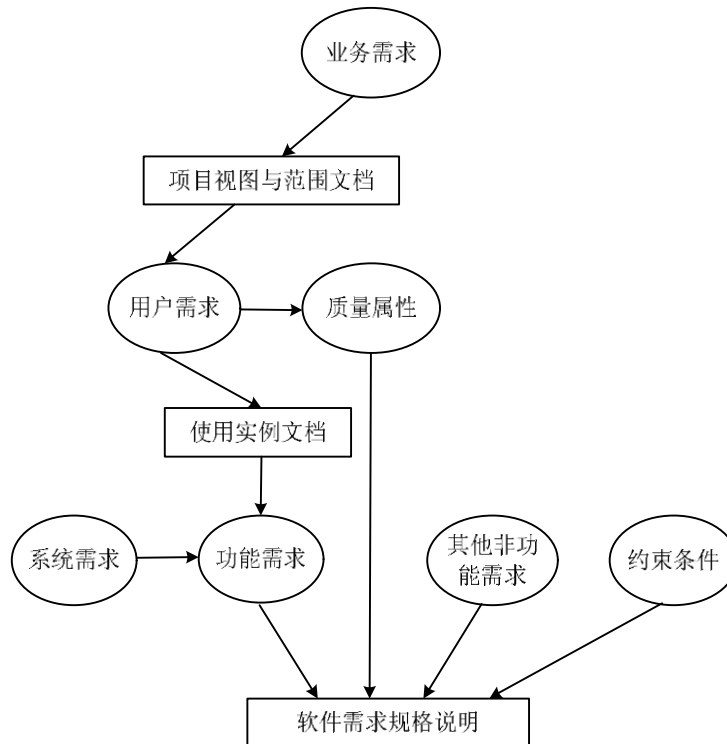


图 3-1 软件需求各组成部分之间的关系

### 3.1.3 需求分析的任務

軟體需求分析是在軟體計畫的基礎上進行的。需求分析的任務就是借助於當前系統的邏輯模型匯出目標系統的邏輯模型，確定目標系統“做什麼”的問題。需求分析的結果應該反映的是系統必須做什麼，而不是怎麼做。具體地講，該階段的工作是在對問題進行調查瞭解的基礎上，確定系統的目標和範圍，將使用者的需求變為軟體的功能和性能描述。為了將軟體的功能和性能描述清楚，系統分析人員需要用一定的方法和手段對問題進行分析，建立反映問題所涉及的資訊、功能及系統行為的模型，用文字、圖形符號來詳細說明軟體必須要做什麼以及配合運行的環境應該是什麼，形成需求規格說明。

需求分析是介於系統分析和軟體設計階段之間的橋樑。一方面，需求分析以系統規格說明和專案計畫作為分析活動的基本出發點，並從軟體角度對它們進行檢查與調整；另一方面，需求規格說明又為使用者和開發人員提供一起協商討論的基礎，作為軟體設計、實現、測試和維護的依據。良



好的分析活動有助於避免或儘早剔除早期錯誤，從而提高軟體生產率，降低開發成本，改進軟體品質。

隨著軟體系統規模的擴大，需求分析與定義在整個軟體發展與維護過程中越來越重要，直接關係到軟體的成功與否。人們逐漸認識到需求分析活動不僅限於軟體發展的最初階段，它貫穿於系統開發的整個生命週期，並將需求工作分為需求開發和需求管理兩部分，將這兩部分統稱為“需求工程”（Requirement Engineering, RE）。20 世紀 80 年代中期，需求工程成為軟體工程的子領域，進入 90 年代以來，需求工程成為研究的熱點之一。需求工程結構圖如圖 3-2 所示。

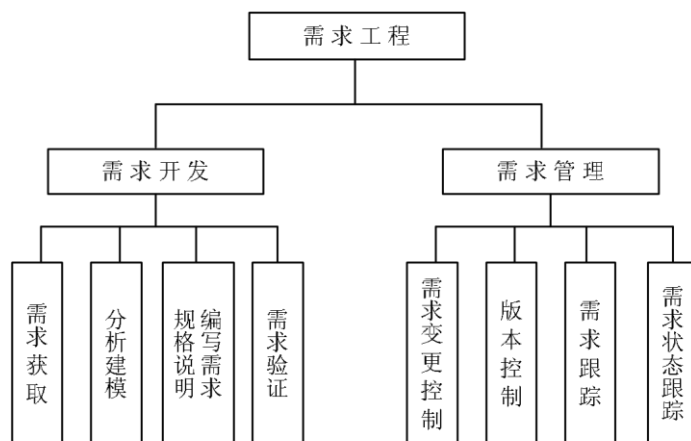


图 3-2 需求工程结构图

由此，需求分析階段的任務就是實現需求工程，具體內容如下。

### 3.1.3.1 需求開發

需求開發工作包括軟體產品的需求收集、評價、編寫文檔等所有活動，分為 4 個階段：需求獲取、分析建模、編寫需求規格說明、需求驗證。

#### 1. 需求獲取

進行用戶需求調查，獲取需求，識別問題，確定系統的綜合要求是需求分析的第一步，是一個對所求解問題及其環境的理解、分析和綜合的過程，也是進行軟體設計與實現的基礎。

在系統開發階段的初期，分析人員往往對待解決的問題知之甚少，而用戶對問題的描述、對目標軟體的要求通常也相當零亂、模糊。尤其是分析人員與用戶共同的知識領域不多，造成相互之間理解方面的困難。分析員通過與用戶充分交流，準確、完整地獲取使用者需求，確定軟體系統的綜合要求。通常軟體系統的綜合要求包括以下幾方面：

(1) 系統功能要求：分析使用者要求實現的全部功能，並分析其中每個要求的必要性和相容性。確定系統應該做什麼，系統要求輸入什麼資訊，輸出什麼資訊，以及如何將輸入變換為輸出，劃分出目標系統必須完成的所有功能。

(2) 性能要求：理解使用者對系統性能的要求，確定目標系統必須達到的技術性能指標。如回應時間，存儲容量及後援存儲，計算精度與效率，系統安全指標等。

(3) 運行和擴充要求：合理地規定系統運行要求和系統將來可能的擴充要求。



系統運行要求通常包括系統運行的物理環境，如系統運行的設備地點，位置是集中式的還是分散式的，對環境的要求如何（如溫度、濕度，電磁場干擾等）；支援軟體系統；資料通信方式；系統介面，如要求與其他系統進行資料交換的內容與格式、終端使用者的類型與熟練程度、使用者對介面的特定要求、使用者操作的易接受性等。

系統可能的擴充要求主要有是否要求可移植、未來擴充或者升級的要求、擴充的方式、範圍、介面等。

（4）系統維護要求：包括系統出錯後可以允許的最大恢復時間、對錯誤修改的回歸測試要求、系統運行日誌規格、是否允許對系統修改、系統變化如何反映到設計中等。（5）系統文檔規格要求：系統要求交付什麼文檔及各類文檔的編制規範和預期使用物件等。

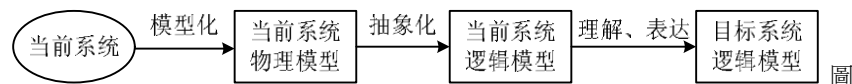
獲取需求的方法見 3.2 節。

## 2. 分析建模

為了更好地識別問題，應綜合上述分析的結果，對已獲取的需求進行抽象描述，為目標系統建立一個詳細的邏輯模型。模型是形成需求說明的重要工具，通過模型可以更清晰地記錄使用者對需求的表達，更方便地與用戶交流，以便幫助分析人員發現用戶需求中的不一致性，排除不合理的部分，挖掘潛在的使用者需求，確定被開發系統的運行環境、功能和性能要求。

建模方法有多種類型，如面向過程的方法、面向資料結構的方法、面向資料流程的方法及物件導向的方法等。本書後面將要介紹的分析方法，實際都是建模方法。通常系統邏輯模型可用資料流程圖、實體—聯繫圖、狀態轉換圖、資料字典和主要功能的處理演算法等進行描述。

通常，軟體發展是要實現系統的物理模型，即確定待開發的軟體系統的系統元素，然後把功能和資料結構分配到這些系統元素中。但是，目標系統的具體物理模型是由它的邏輯模型經產生實體得到的，所謂邏輯模型就是忽略實現機制和具體細節，只描述系統要完成的功能和要處理的資料。因此，軟體系統的開發，可看做是建立模型及模型轉換的過程。具體分析建模過程如圖 3-3 所示。



3-3 分析建模過程

## 3. 編寫需求規格說明

該階段的主要工作是需求描述。在對問題空間準確、全面理解的基礎上，對需求模型進行精確地、形式化的描述。需求描述應定義和詳細描述全部系統功能，反映影響系統事件前後關係的軟體行為，建立系統介面的特徵，並揭示設計限制。結果以文檔形式表述，成為可見的，能夠與用戶交流的，可以進行複審的系統邏輯模型。該階段的文檔包括：需求規格說明書、使用者手冊初稿、確認測試計畫、修改的開發計畫等。其中：

（1）需求規格說明包含對目標軟體系統的外部行為的完整描述、需求驗證標準以及使用者在性能、品質、可維護性等方面的要求。

（2）使用者手冊包括使用者介面描述以及有關目標系統使用方法的初步構想。在需求分析階段就開始編寫用戶手冊，而不是在編碼測試完成後組織人員另行編寫，其優點在於，首先，使系統



分析和設計人員在系統開發的早期就從用戶的角度觀察和分析系統，有利於提高系統對使用者的友好程度，並有利於提高系統運行的方便性和實用性；其次，可以使使用者手冊隨著系統各階段的開發不斷完善，有利於保證其正確性、完備性和同真實系統的一致性，並可保證其按時完成。

(3) 在需求分析中確立測試標準，作為系統開發目標是否完成的驗收依據。而該測試計畫的依據是系統目標，包括系統功能和性能等詳細內容的使用者需求。由於系統目標是在需求分析階段制定的，所以，以此為依據的確認測試計畫應在同一階段，由相同的人員制定。這樣能最有效地保證兩者的一致性。

(4) 修改的專案開發計畫是根據新的分析結果，對可行性分析和軟體計畫階段中制定的初步的專案開發計畫作必要的修改、補充和完善。

#### 4. 需求驗證

在以上各階段的實施過程之中，形成了有關的文檔。在此基礎上，由專家、分析人員、開發人員、用戶組成評審組，對需求分析所得結果的正確性、合理性和有效性進行檢查，以確保需求分析的全面性、準確性和一致性。並使使用者和開發人員對需求規格說明及使用者手冊達成一致的理解。通常應從以下幾點進行評審：

(1) 完整性：完整性體現在兩個方面。首先是不能遺漏任何必要的需求。避免遺漏需求的關鍵是需求獲取的方法，後面將詳細討論這個問題。需求完整性的第二層含義是清楚、完整地描述每一項需求所要完成的任務，使開發人員理解實現這項需求的所有必要資訊，使用者能夠審查這項需求描述的正確性。

(2) 正確性：所謂正確性是指每項需求都必須準確地反映用戶要完成的任務。不僅要從不同角度檢查需求的正確性，還應該檢查每項需求是否與軟體的總體目標一致，是否超出了業務需求所定義的軟體範圍。

(3) 一致性：用戶需求必須和業務需求一致，功能需求必須和用戶需求一致。嚴格地遵守不同層次間的一致性關係，就可保證最後開發出來的軟體系統不會偏離最初的實現目標。

(4) 必要性：必要性即每項需求都應該是客戶所需要的，開發人員不得自作主張添加需求。檢查需求必要性的方法是將每項需求回溯至用戶的某項輸入。

(5) 無歧義性：無歧義性即需求分析結果的描述，應使不同的人員對需求的理解是一致的。如採用自然語言描述需求，其優點是任何人未經訓練，都可理解，但自然語言對需求分析最大的弊病就是它的二義性。所以各種需求分析方法中都對描述語言作了一些限制，對描述語言的規定是需求分析方法的重要組成部分。發現需求二義性可通過對需求文檔的正規檢查，包括編寫測試用例和開發原型，還可使用按多種不同的方式、從多個角度描述同一需求的方法發現需求二義性。

(6) 可驗證性：每項需求都應該是可驗證的。系統分析員在需求分析時就要考慮每項需求的可驗證性問題，為需求設計測試用例或其他的驗證方法。

(7) 優先順序的劃分：為每一項需求按照重要程度分配一個優先順序，在開發產品時，可以先實現優先順序最高的核心需求，將優先順序低的需求放在後續版本中。優先順序的劃分可以幫助





專案管理者解決衝突、安排階段性交付，在必要時做出功能取捨，以最少的費用獲得軟體產品的最大功能。

在評審中，一旦發現問題，應儘快予以更正。若需求分析工作通過評審，則可以開始下一階段工作。同時，需求規格說明成為使用者方與系統開發方之間的合同，任何增刪或改動所引起的開發規劃及成本變化，應由提出方承擔責任。若未通過評審，則需重新進行有關的需求分析工作。

需求開發的過程如圖 3-4 所示。

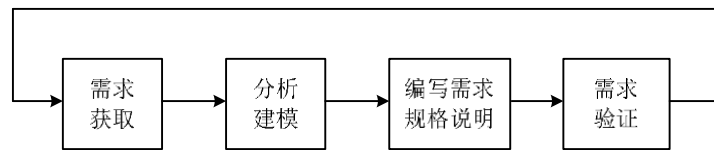


图 3-4 需求开发的过程

### 3.1.3.2 需求管理

由於基於電腦系統的需求常常要變更，而且變更需求的要求貫穿於整個生命期，所以，完成需求開發，形成規格說明僅是需求成功的一半，開發人員必須能夠真正把所有客戶的需求應用到產品中，並能夠有效地控制需求變更，才能保證需求與設計的一致性，最準確地實現既定的需求。需求管理就是一組在軟體發展進展中的任何時候標識、控制和跟蹤需求的活動。

需求管理的內容包括在工程進展過程中為保證需求集成性及精確性所進行的所有活動，具體內容包括需求變更控制、版本控制、需求跟蹤和需求狀態跟蹤。

(1) 需求變更控制：建議變更；評審所提出的需求變更，評估分析每項變更可能的影響，從而決定是否實施變更；以一種可控制的方式將需求變更融入到專案中。

(2) 版本控制：建立需求基準版本和需求控制版本文檔，確定一個需求基準。每個版本的需求規格說明都必須是獨立說明，以避免將底稿和基準或新舊版本相混淆。需求文檔的每一個版本必須被統一確定，在變更時，應記錄變更需求文檔版本的日期以及所做的變更、原因，還包括由誰負責更新和更新的新版本號等。可採用版本控制工具自動完成這些任務。

(3) 需求跟蹤：跟蹤所有受需求變更影響的工作產品，讓每項需求都能與其對應的設計、原始程式碼和測試用例聯繫起來。當進行某項需求變更時，與其相關部分可能也需要修改。通過需求跟蹤所建立的聯繫，可方便地找到相關的其他需求、設計範本、原始程式碼和測試用例。通過跟蹤減少當變更需求時必須進行的變更被遺漏，確保覆蓋全部的需求，同時確保所有的輸出符合用戶的需求。

(4) 需求狀態跟蹤：定義需求狀態，在整個專案過程中跟蹤需求的每一個狀態及其變更情況。可將跟蹤每項需求的狀態建立一個資料庫，其中每一條記錄保存一項功能需求的重要屬性。需求狀態有已推薦的、已通過的、已實施的或已驗證的等，這樣在任何時候都能得到每個狀態類的需求數量。

需求開發的結果是形成了客戶與開發人員雙方均滿意的系統邏輯模型，它連接需求開發和需求管理，作為需求管理的輸入。需求管理的過程，從需求獲取即開始，並貫穿於整個軟體專案生命週期，以實現最終產品同需求的最佳結合。



需求開發與需求管理間的關係可用圖 3-5 來表示。

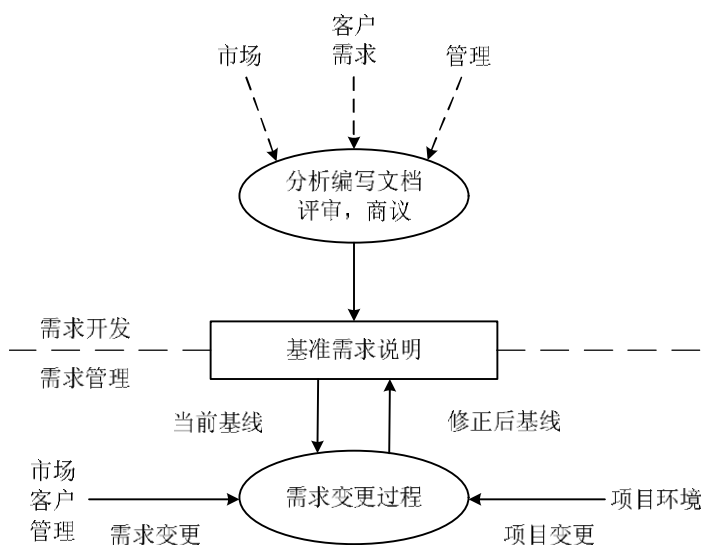


图 3-5 需求开发与需求管理间的关系

## 3.2 獲取需求的方法

綜上所述，需求分析的主要任務，就是正確理解和表達用戶的要求，但如何從應用領域獲取所需知識，往往造成需求分析的障礙，所以獲取需求是需求分析的關鍵的一步，也是最困難、最易出錯的一步。

### 3.2.1 存在問題

在獲取需求過程中遇到的典型問題有：

- (1) 對需求的理解問題。要準確、完整地獲取需求必須對問題進行深入的理解與把握。而大多數情況下，應用領域具有一定的專業性，分析人員不是問題領域的行家，造成理解問題。
- (2) 分析人員與用戶的通信問題。由於分析人員不懂特定的業務，需要通過使用者的描述來瞭解。而用戶對應用問題的理解、描述以及他們對目標系統的要求往往具有片面性、模糊性，在很多情況下，使用者往往不能正確表達他們的需求。而且用戶多是考慮業務領域本身，而分析員對問題的理解必須從資訊處理要求出發，所以與用戶建立相互信任，有效地溝通是分析員的首要任務。
- (3) 用戶需求的可變性問題。應用領域與用戶需求具有多樣性，由於用戶領域的業務不斷擴展或者轉移、市場競爭的要求或用戶主管人員的變更等原因，使得用戶的需求常發生變化。需求的可變性就要求分析員能夠使其工作適應需求的變化，給需求分析造成了很大的困難。
- (4) 分析方法和分析工具問題。需求分析方法論和分析工具的缺乏，及其應用範圍的局限性是造成障礙的又一原因。



### 3.2.2 常用方法

需求獲取方法是溝通用戶和開發人員之間的橋樑。目前，需求分析方法中，用戶需求獲取主要是依靠以下幾種方法：

#### (1) 訪談。

訪談是最早開始使用的獲取用戶需求的方法，也是目前仍然廣泛使用的需求分析技術。

訪談有兩種基本形式，分別是正式的和非正式的訪談。正式訪談時，系統分析員將提出一些事先準備好的具體問題，例如，詢問處理的單據種類、處理的方法以及資訊回饋時間應該多快等。而在非正式訪談中，分析員可提出一些用戶可以自由回答的開放性問題，例如，詢問用戶對目前正在使用的系統有哪些不滿意的地方，以鼓勵被訪問人員說出自己的想法。詢問一個開放的、可擴充的問題將有助於更好地理解用戶目前的業務過程，並且確定在新系統中應如何解決目前系統的問題。分析人員通過用戶對問題的回答獲取有關問題及環境的知識，逐步理解使用者對目標系統的要求。

採用訪談方式時分析員的主要任務是問題的設計，包括探討功能、非功能、例外情況的問題，甚至一些看起來“愚蠢”的問題。必須把所有的討論記錄下來，同時還要做一定的整理，並請參與討論的用戶評論並更正。

#### (2) 問卷調查。

問卷調查即把需要調查的內容製成表格交給使用者填寫。該方法對需要調查大量人員的意見時，十分有效。這種方法的優點是：用戶有較寬裕的考慮時間和回答時間。經過仔細考慮寫出的書面回答可能比被訪者對問題的口頭回答更準確，從而可以得到對提出的問題較為準確細緻的回答。分析員仔細閱讀收回的調查表，然後再有針對性地訪問一些用戶，以便向他們詢問在分析調查表時發現的新問題。

採用問卷調查方法的關鍵是調查表的設計。在開發的早期用戶與開發者之間缺乏共同語言，使用者可能對表格中的內容存在理解上的偏差。因此調查表的設計應簡潔、易懂、易填寫，同時還要注意用戶的特點和調查的策略。

#### (3) 情景分析。

由於很多使用者不瞭解電腦系統，對自己的業務如何在將來的目標系統中實現無認識，所以很難提出具體的需求。所謂情景分析就是對目標系統解決某個具體問題的方法和結果，給出可能的情景描述，以獲知用戶的具體需求。

情景分析技術的優點是，它能在某種程度上演示目標系統的行為，便於使用者理解，從而進一步揭示出一些分析員目前還不知道的需求。同時，讓用戶起積極主動的作用對需求分析工作獲得成功是至關重要的，情景分析較易為用戶所理解，使得用戶在需求分析過程中能夠始終扮演一個積極主動的角色。因此在訪問用戶的過程中使用該技術是非常有效的。

#### (4) 實地考察。

分析人員到用戶工作現場，實際觀察用戶的手工操作過程也是一種行之有效的需求獲取方法。

在實際觀察過程中，分析人員必須注意，系統開發的目標不是手工操作過程的模擬，還必須考慮最好的經濟效益、最快的處理速度、最合理的操作流程、最友好的使用者介面等因素。因此，分析人員在接受用戶關於應用問題及背景的知識的同時，應結合自己的軟體發展和軟體應用經驗，主





動地剔除不合理的、一些暫時行為的使用者需求，從系統角度改進操作流程或規範，提出新的潛在的用戶需求。

#### (5) 構造原型。

在系統開發的早期，以對用戶所進行的簡單需求分析為基礎，快速建立目標系統的原型。用戶可以通過原型進行評估並提出修改意見，從而使用戶明確需求。快速原型方法既可針對整個系統，也可針對系統的某部分功能。(快速原型方法內容詳見 3.4 節)。

### 3.2.3 需求分析的原則

需求分析應遵循以下原則：

#### (1) 解決邏輯問題。

需求分析是對問題的識別和說明過程，分析員要回答的是“系統必須做什麼”的問題，而不是“系統應該怎麼做”的問題。需求分析的基本原則是給出要完成的功能和處理的資訊，而不考慮實現的細節，即需求分析工作集中在系統應當完成什麼功能上，而不在怎樣才能實現這些功能上。

#### (2) 以運行環境為基礎。

需求分析工作必須以具體的運行環境為基礎，系統分析人員可以參考，但不能照搬其他類似的系統開發時的分析工作，更不能憑個人的好惡或主觀想像辦事。

#### (3) 用戶參與的原則。

需求分析工作是系統分析人員同使用者不斷交互的過程。因此，需求分析工作應該要有客戶(開發委託者或開發委託者兼系統使用者)所指定的人員參加，以保證交互的充分性和工作效率。

#### (4) 構造高品質的需求規格說明。

需求規格說明是需求分析工作重要的完成標誌。在生成規格說明的有關文檔的過程中，分析人員應該嚴格遵循既定規範，做到內容全面、結構清晰、格式嚴謹。

### 3.2.4 需求分析方法概述

隨著軟體發展技術的發展，目前已形成許多需求分析及描述的方法，每種方法都有其獨到之處。但不管採用哪種方法進行需求分析，都應滿足如下基本要求：

#### (1) 必須能理解問題的資料欄和功能域。

一個軟體從外部可以被看成是一個黑盒子，資訊從一端流入，從另一端流出。在內部對輸入資訊進行轉換，形成中間結果，再轉換，最終得到所需要的輸出資訊，就是軟體所具有的功能。所以可通過對資料的描述和資料轉換的描述，實現對系統的理解和描述。

電腦程式所處理的資料欄的描述一般為：資料內容、資料結構和資料流程。資料內容就是資料項目，資料結構就是資料項目的組織形式，資料流程是資料通過系統時的變化方式。

對資料進行的一系列的轉換即系統應實現的功能和子功能。兩個功能之間的資料傳遞就確定了功能之間的介面。在需求分析階段的功能描述通常是用文字說明要“做什麼”，不必具體展開怎樣做。

#### (2) 必須能按自頂向下、逐層分解的方式對問題進行分解和不斷細化。



現實世界是複雜多變的，從整體上考慮一個問題通常是困難的。特別是一些複雜的問題，作為一個整體很難準確理解，不可能給出正確的解決方案。此時，對問題進行分解與抽象是普遍有效的基本法則。

分解是將求解的複雜問題，分解為若干相對簡單問題求解的組合。例如實現一個教學管理系統，可以將該系統分解為學生管理、教師管理、課程管理、教務管理、考試管理等 5 個子系統。定義好各子系統之間的相互聯繫，對每個子系統分別求解。如子問題仍然較複雜，則可以進一步分解。如可將考試管理進一步分解為試題庫維護、試卷生成、考務管理、學生考試和評閱試卷等子系統。

分解的目的是為了降低問題求解的複雜性，將複雜問題分解成一些小的、容易控制和理解的子問題不僅便於理解，還可以將子問題劃分給不同的開發小組，分別完成，然後再裝配起來形成一個完整的系統。更重要的是通過分解，可以促進軟體發展走向構件開發的道路。因為劃分的小問題中有些是常見的公共問題，有些是特殊的問題。對常見的公共問題可以複用已有的軟體構件，開發人員只對特殊的問題提供解決方案。這樣不僅可提高軟體發展的效率，還使軟體發展向著“工程化”方向邁進。

在需求階段分解的內容，可以是軟體的功能域和資料欄兩個部分，分解的方式有橫向分解和縱向分解。橫向分解在同一層次上，把一個功能分解為幾個子功能，並確定這些子功能與父功能之間的介面；縱向分解是將一個功能分解為幾個子功能，然後對其中複雜的子功能再繼續分解為更小的子功能，逐層分解下去直到獲得合適的子功能為止。在實際操作中，沒有單純的橫向分解或縱向分解，通常是二者相結合的分解方式，如圖 3-6 所示。

而抽象是認識問題的一般與特殊的關係。例如在上述的考試管理子系統中，可以考慮考試要求的不同試題類型，構造每種類型的典型試題，通過對典型試題的答題要求和閱卷判定方法分析，抽象出各類試題的不同答題模式和電腦閱卷策略與演算法。

問題分解與抽象定義了問題的層次結構，應該在問題求解中反映出這種層次結構。結構與問題求解結構的對應關係保證了問題定義的完整性、正確性和可跟蹤性。

(3) 要給出系統的邏輯視圖和物理視圖。

軟體需求的邏輯視圖描述的是軟體要達到的功能和要處理的資訊之間的關係，但沒有描述實現的細節。如庫存系統中的檢查庫存的功能，在邏輯視圖中只關心庫存檔的資料結構，而不考慮電腦的具體存儲方式。軟體需求的邏輯描述是軟體設計的基礎。

軟體需求的物理視圖給出的是處理功能和資訊結構的實際表現形式，需考慮實際的環境和具體的設備。如一些資料是由終端鍵盤輸入的，而有些資料可能是由模一數轉換設備提供的。軟體分析人員必須弄清已確定的系統元素對軟體的限制，並考慮功能和資訊的物理表示。注意此時功能和資訊的物理表示只限於“系統必須做什麼”的範圍，而不考慮“如何做”的細節。

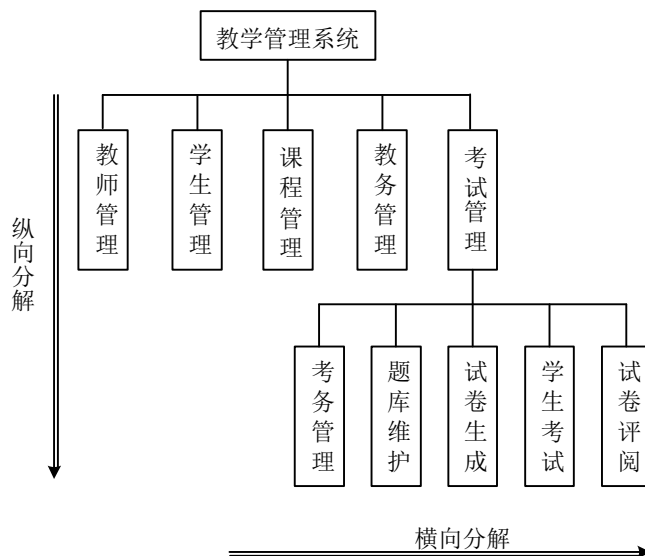


圖 3-6 問題的分解

由上述討論可見，系統的邏輯視圖和物理視圖描述了系統滿足處理需求所提出的邏輯限制條件和系統中其他成分提出的物理限制條件。

### 3.3 結構化分析方法

結構化方法是 20 世紀 70 年代初，由 E.Yourdon、L.Constantine、T.DeMarco 等人提出的一種系統的軟體發展方法，包括結構化分析（SA）、結構化設計（SD）和結構化程式設計（SP）。結構化分析方法，多年來被廣泛應用，是最經典的面向資料流程的需求分析方法。適用於分析大型的資料處理系統。

#### 3.3.1 結構化分析方法的基本思想

SA 方法以資料流程分析作為需求分析的出發點，任何資訊處理過程均看作是將輸入資料變換成所要求的輸出資訊的裝置。而當分析人員面對一個複雜的問題時，結構化分析的策略是基於問題分解與抽象的觀點，用抽象模型概念，按照軟體內部的資料傳遞關係，採用自頂向下、逐層分解技術，直至找到滿足功能需求的可實現軟體元素為止。

該方法的特點是利用資料流程圖來幫助人們理解問題，對問題進行分析，即利用圖形工具來類比資料處理過程。該方法的核心是資料流程圖。資料流程圖是一種用來表示資訊流程和資訊變換過程的圖解方法，它將系統看成是由資料流程聯繫的各種功能的組合。資料流程圖可以方便地描述由資料流程的流動聯繫的各種功能。通過各種功能的輸入/輸出結果，表現現有系統或待開發系統的功能。



具體做法是首先將整個系統看作一個加工資訊處理的裝置，是一個黑匣子，標識出系統邊界和所有輸入輸出資料流程。然後自頂向下，對加工內部進行細化，逐層分解，將複雜功能分解為若干簡單功能的有機組合，繪製資料流程圖，並逐步補充細節描述。通過將系統分解成多層處理後，在較低層次上，可以看到資料流程圖的高層次加工的細節和相關的資料流程。用資料字典精確定義資料流程圖中每個資料流程的成分及每個成分的屬性、資料結構、數量、傳遞方式等。用結構化英語、決策樹和判定表對資料流程圖中的每個加工的功能進行描述。

結構化分析方法的實質就是一種強烈依賴資料流程的自頂向下的建模方法，採用一組分層的資料流程圖及相應的資料字典作為系統的邏輯模型。它不僅是需求分析技術，也是完成規格說明文檔的技術手段。

### 3.3.2 描述工具

SA 方法提供一套圖形、表格和結構化語言等半形式化的描述方式表達需求，簡明易懂。描述工具包括：

(1) 資料流程圖 (Data Flow Diagram, DFD)：描繪系統邏輯模型的圖形工具，描述了系統的組成部分及各部分之間的聯繫。通常通過對系統的分解得到一套分層的資料流程圖。

(2) 資料字典 (Data Dictionary, DD)：DFD 只描繪資訊在系統中的流動和處理情況，而資料字典則是對圖中的元素進行定義。

(3) 結構化英語、判定表和決策樹：詳細描述資料流程圖中一些複雜處理的加工邏輯。

### 3.3.3 資料流程圖

SA 方法使用資料流程圖從資料傳遞和加工的角度，以圖形的方式刻畫資料流程從輸入到輸出的傳輸變換過程。資料流程圖是結構化系統分析的主要工具，它表示了系統內部資訊的流向，並表示了系統的邏輯處理的功能，是一種功能模型，如圖 3-7 所示。

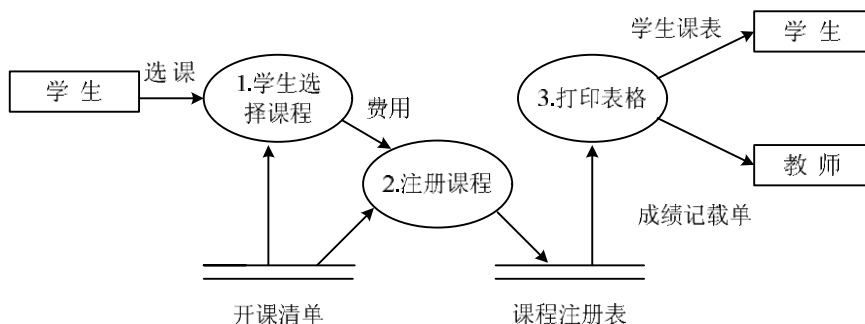


图 3-7 一个学生选课系统的 DFD

#### 3.3.3.1 基本符號

資料流程圖中的基本圖元包括：

○：圓框，表示加工。

□：方框，表示資料的源點或資料的終點。



→：箭頭，表示被加工資料的路徑和流向，即資料流程。

=：雙杠，表示資料存儲，可以是一個記錄或一個資料檔案，可用名詞或名詞性短語命名。

### 1. 資料來源點和數據終點

資料來源點和資料終點用方框表示，它是系統之外的實體，可以是人、物、部門或者其他系統，如圖 3-7 中的教師和學生。資料來源點是資料流程的起點，資料終點是系統資料流程的最終目的地。利用資料來源點與終點明確標識出系統與環境的介面，給出系統有效作用範圍的邊界。

### 2. 加工 (資料處理變換)

加工用圓框表示，是對資料進行處理的邏輯單元。它接受若干輸入資料流程，通過加工內部產生規定的輸出資料流程。資料流程圖中對加工的標識通常由加工編號和加工命名組成。如圖 3-7 中的學生選擇課程、註冊課程和列印表格都是加工的例子。為了給讀者理解系統提供有意義的資訊，對加工的命名通常要求使用“動詞+賓語”形式的結構化語言簡單明確地進行標識。例如“註冊課程”，“列印報表”等，不應使用如“計算”、“處理”、“加工”等抽象名詞作為加工命名。如果一個加工很難給出適當的命名，應該考慮該加工的處理功能是否恰當，是否應該重新分解。

### 3. 資料流程

資料流程用帶資料流程標識的箭頭表示，表示系統處理的資料物件和資料流程動的方向。資料流程的方向可以是：從一加工流向另一加工；從加工流向資料存儲或資料存儲流向加工；從源點流向加工或從加工流向終點。

當資料流程的方向指向一個加工時，表示它是該加工的一個輸入資料流程；當資料流程的方向是從一個加工發出時，表示它是通過該加工得到的一個輸出資料流程。資料流程是客觀世界的實體物件的邏輯表示，可以是一個資料項目，也可以是一組資料項目組成。例如圖 3-7 中“費用”由“學生學號”、“註冊課程號”和“金額”組成。一個資料項目可以是基本項（不可再分解的資料項目，如組成“費用”的資料項目），也可以是結構型資料項目。例如“選課單”由“學生學號”和“註冊課程列表”組成，“註冊課程列表”由若干“註冊項”構成，每個“註冊項”由“課程名”、“課程號”、“學分”構成。

每個資料流程應有良好的命名，它不僅作為資料的標識，還有利於深化對系統的認識。可以考慮使用資料流程中最主要的資料項目作為該資料流程的名稱。與加工命名類似地，應該避免使用諸如“資料”、

“資訊”之類地抽象名詞標識資料流程。因為加工是一個進行資料處理的黑匣子，因此流入、流出同一加工的資料流程應該具有不同的名稱。資料流程不應包括控制流。資料流程反映的是加工處理的物件，控制流是一種選擇或用來影響加工的性質，而不是對它進行加工的物件。例如圖 3-8 中的“讀下張卡片”即屬於控制流，不應該在資料流程圖中畫出。



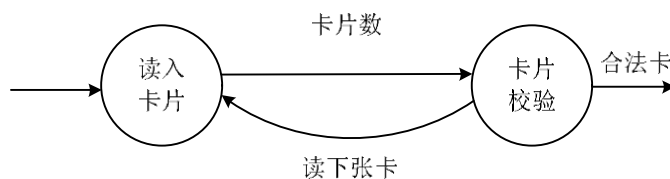


图 3-8 含有控制流的例子

#### 4. 資料存儲

資料存儲用兩條平行線段表示，邏輯上是資訊的靜態存儲。物理上，DFD 中的資料存儲可以是電腦系統中的外部或者內部檔、檔的一部分、資料庫的元素或記錄的一部分等，還可以是一個人工系統中的表冊、帳單等。資料存儲是系統的重要組成部分，在分層資料流程圖中，通常是局部於某一分解層次的。資料存儲可用名詞或名詞性短語命名，還需要在資料詞典中說明其邏輯或者物理組織要求以及存儲介質等。一個資料流程從加工流向資料存儲，表示該加工對檔寫；如果資料流程是從資料存儲流向加工，表示該加工對檔讀。如果加工到資料存儲之間的資料流程是雙向的，表示該加工對檔的操作包括讀、寫和修改。流入、流出資料存儲的資料流程的名字通常和資料存儲同名，可以不標識。

##### 3.3.3.2 資料流程與加工之間的關係

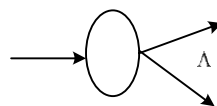
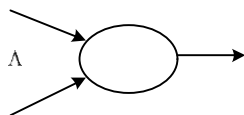
在資料流程圖中，兩個加工之間可以有多個資料流程。如果有兩個以上的資料流程指向同一加工，或一個加工流出兩個以上資料流程，則這些資料流程之間往往存在一定的關係。為表達資料流程之間的邏輯聯繫，可以附加說明標記符號。常見的說明符號及含義如圖 3-9 所示。

圖 3-9 (a) 表示流入加工的兩個資料流程必須同時到達，該加工才能啟動。

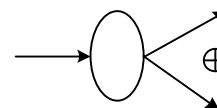
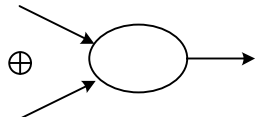
圖 3-9 (e) 表示流入加工的兩個資料流程只要有一個到達，該加工即可啟動。

圖 3-9 (c) 表示流入加工的兩個資料流程不能同時到達。

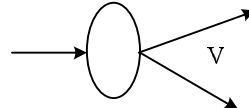
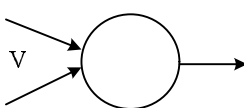
對於圖 3-9 (b)、圖 3-9 (d)、圖 3-9 (f) 則是對流出加工的資料流程相應關係的對應解釋。



(a) 流入加工的兩個資料流程的聯繫為與 (b) 流出加工的兩個資料流程的聯繫為與



(c) 流入加工的兩個資料流程的聯繫為異或 (d) 流出加工的兩個資料流程的聯繫為異或



(e) 流入加工的兩個資料流程的聯繫為或 (f) 流出加工的兩個資料流程的聯繫為或



圖 3-9 流入/流出同一加工資料流程

### 3.3.3.3 資料流程圖的分層

對於一個大型軟體系統，不可能將全部最終的加工和資料流程都在一張圖上表現出來。這樣圖面太大，關係複雜，難以理解。結構化控制複雜性的方法，是採用分層技術，用一套分層 DFD 來分解複雜性。分層體現了抽象和資訊隱蔽，即上層不考慮下層的細節，暫時掩蓋了下層加工的功能及它們的複雜關係。

一個軟體系統的一套分層 DFD 圖包括頂層 DFD、中間層 DFD 和底層 DFD 組成。頂層圖只有一張，它描繪了整個系統的作用範圍，可將整個系統作為一個加工，其加工名就是系統名，輸入和輸出就是系統的所有輸入和輸出資料，也就是系統與外界介面；中間層的 DFD 圖是對上層父圖的分解，它的每一加工還可繼續分解細化。當分解一直進行到每個加工的功能獨立，簡單明確，資料流程被嚴格定義時，即得一組底層 DFD 圖。每張底層 DFD 圖是由一些不能再分解的加工和簡單資料流程組成，這些加工被稱之為基本加工。顯然，這種自頂向下，逐層地理解和表達系統的分層 DFD 圖，是一個控制複雜度，保證分析品質的很好的系統分析方法。

分解示意圖如圖 3-10 所示。

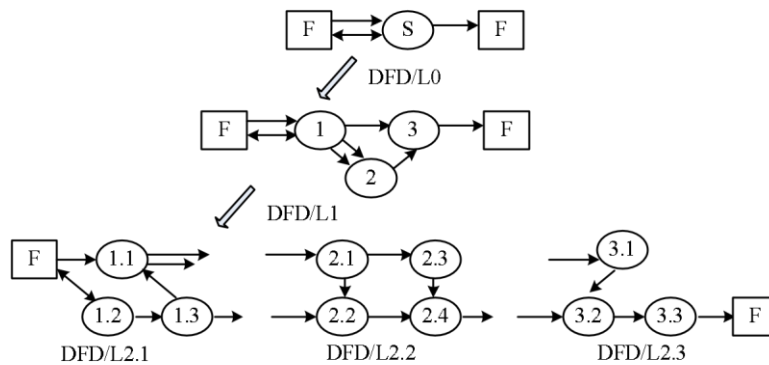


图 3-10 分层数据流图

為了控制分解過程，嚴格地表現一套 DFD 圖，引入如下概念：

(1) 父圖與子圖。如圖 3-11 所示的頂層圖是整個系統的抽象表示，如果系統 S 可分解為三個子加工 S1、S2、S3，畫出相關的資料流程，得到頂層下的第一層資料流程圖。繼續分解 S1、S2、S3，得到多層的資料流程圖。其中上一層圖是其分解的下一層圖的父圖。相反下一層圖是上層圖的子圖。父圖裡的加工是其相應子圖的抽象表示，子圖則是其父圖中相應加工的細化。

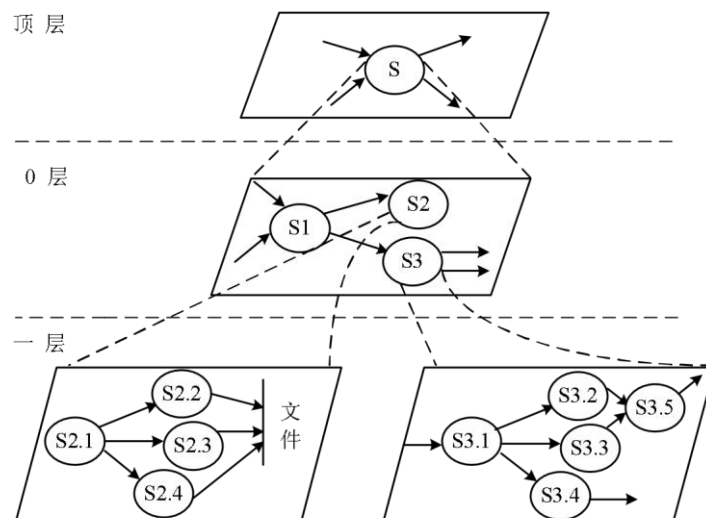


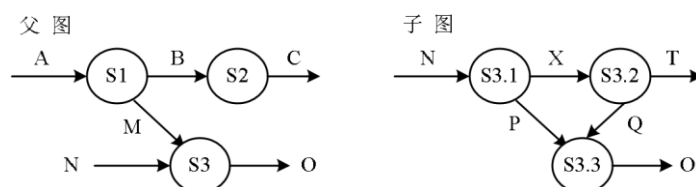
图 3-11 分解示意图

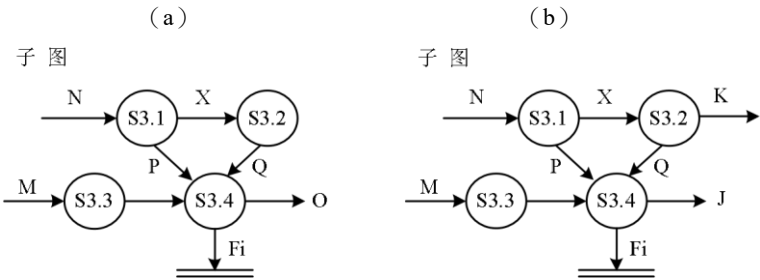
(2) 分層圖編號。頂層圖中加工，即整個系統的編號以 0 表示，以下每一張子圖的編號就是上圖中相應加工的編號，而子圖中每個子加工的編號應該是子圖號，加小數點，加局部加工號。在一張分層圖中，每個加工的編號中所含小數點的個數，就是該圖的層次數。圖 3-11 顯示了這一編號規則。

(3) 父圖和子圖資料流程的平衡性。在繪製分層 DFD 圖時，必須注意子圖與其父圖中相應加工的資料介面保持平衡，即父圖中流入流出某加工的資料流程，與分解後子圖中流入流出加工的資料流程應保持邏輯上的一致。控制分層 DFD 圖的資料流程的平衡性，是消除 DFD 圖錯誤的一個重要手段。例如圖 3-12 (b) 是圖 3-12 (a) 中 S3 的分解，父圖中流入 S3 的資料流程為 M、N，流出 S3 的資料流程為 O，而在圖 3-12 (b) 中流入的資料流程少了 M，顯然是錯誤的。

注意，很多情況下父圖和子圖資料流程的平衡性，不一定是資料流程的名稱和個數的形式一致，如圖 3-12 (d) 所示，流出的資料流程為 J、K，從表面上看，資料流程的名字和數量與圖 3-12 (a) 中流入流出 S3 的均不一致，但若資料流程 J、K 是隨著對圖 3-12 (a) 中加工 S3 的分解，將資料流程 O 分解而來的，則圖 3-12 (d) 與圖 3-12 (a) 仍是平衡的。

(4) 局部檔。如果某個中間層的 DFD 中的資料存儲不是上圖中相應加工的外部介面，而只是本圖中某些加工之間的資訊介面，則稱其為局部檔。一個局部檔只有當它作為某個加工的資料介面或某個加工的特定輸入或輸出時才予以標出。圖 3-12 中的資料存儲 Fi 就是局部檔。局部檔標出的約定有助於資訊隱蔽。





(c) (d) 圖 3-12 父圖和子圖的平衡

3.3.3.4 畫資料流程圖的步驟

構造分層 DFD 圖的過程就是結構化的系統建模的過程，在對分層資料流程圖求精過程中，自頂向下逐層地理解和表達系統。經驗指出，對不同的軟體需要根據不同的思路畫 DFD 圖，但其基本步驟應是：由外向裡，自頂向下，模擬現行的問題處理過程，通過一系列分解步驟，逐步細化、完善求精，最終表現出整個軟體系統的構成。具體步驟如下：

(1) 找外部實體，確定系統邊界，確定資料流程源和資料終點。以專案開發計畫確定的目標為基礎，經過需求獲取工作，可以比較容易劃定系統的邊界，確定系統的資料來源點和終點。進而找出外部實體的輸入和輸出資料流程，畫出頂層資料流程圖。

(2) 從資料來源點出發，按照系統的邏輯需要，逐步畫出一系列邏輯加工框，直至資料終點。自頂向下，對每個加工進行內部分解，畫出分層資料流程圖。

(3) 按照下述一般原則對資料流程圖進行複查求精。複查求精應由分析員與用戶共同參與，分析員借助資料流程圖及資料詞典（詳見 3.3.4 節），向使用者闡述系統輸入資料如何一步一步地轉變為輸出結果。這些闡述集中地反映了分析員當前對於目標系統的認識，使用者應認真聽取分析員的報告，考察處理是否正確，功能是否完整，並及時糾正與補充。分析員由此對資料流程圖進一步求精。

構造分層資料流程圖及求精的一般原則是：

(1) 資料流程圖中所有圖形符號必須是四種元素之一，圖中的每一個元素都必須有良好的命名。資料流程圖的主圖必須封閉在外部實體之間，實體可以有多個。

需要說明的是，在使用中，為方便起見，還常使用另一套等價的符號，兩套符號的對應關係如圖 3-13 所示。

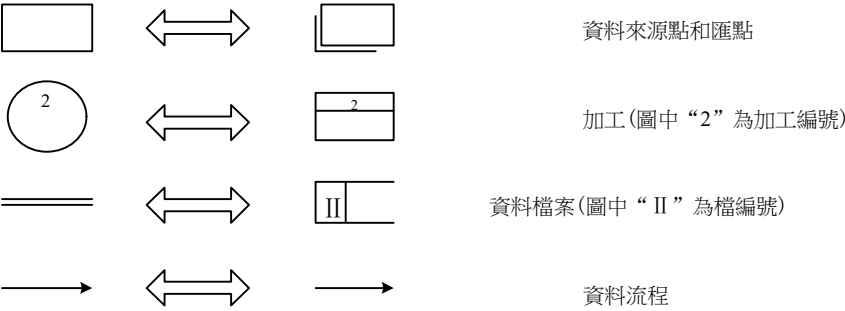


圖 3-13 符号对照



(2) 分解應該自然，概念上清晰、合理。應使每次分解生成的 DFD 上的資料流程和加工都很容易確切地命名，否則說明分解欠合理，應重新考慮。

(3) 流入、流出加工的資料流程應連續。即每個加工必須有輸入、輸出資料流程，僅有輸入（或者僅有輸出）資料流程的加工是不恰當的。同時流入和流出同一加工的資料流程之間應具有對應關係，即每個加工產生的所有資料流程都能夠由進入該加工的資料流程匯出。

(4) 在分解過程中，注意保持父圖和子圖資料流程的平衡性，合理利用局部檔實現資訊隱蔽。

(5) 控制單張資料流程圖的複雜性。資料流程圖的複雜性體現在一張圖上加工和資料流程的數目。一個加工的下層子加工應該控制在 7 個以內。流入、流出同一加工的資料流程也不應太多。若太多，意味著該加工的功能過於複雜，可能是分解不恰當所致。

(6) 當一個加工邏輯足夠簡單，則分解可以終止。常見的終止加工分解的條件如採用結構化語言對加工進行描述時，不超過一頁打印紙；加工只有一個輸入資料流程和一個輸出資料流程等。

(7) 在軟體分析過程中應只考慮穩態，暫時忽略有關細節。如可以暫不考慮初始化、出錯路徑等細節。資料流程圖中不應有控制流和控制資訊，如圖 3-14 所示。其中“ $X \geq 0$ ”即為控制流。

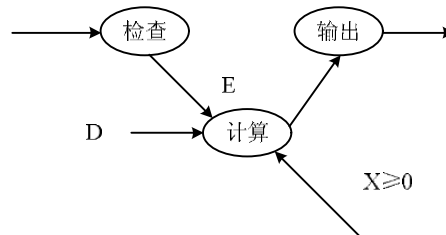


圖 3-14 一個帶有控制流的錯誤 DFD

(8) 在進行逐層分解時，應注意分解層次的均勻性。避免使一次分解的加工太多和太少。因為太多會使一張圖的複雜性增大，太少則會使層次增多，增加了綜合評價的工作量。一般應上層分解快些，下層分解慢些。可以將一個系統的全部分層資料流程圖看作一棵樹。頂層加工為樹根，所有底層加工為樹葉。從樹根到所有樹葉的路徑長度之差不應該太大（理想情況是不超過 2）。兩個底層加工的層次相差太大，說明在某個高層加工的分解可能是不適當的，某些加工已是基本加工，而另一些還須加工分解若干次。

### 3.3.4 資料字典

資料字典是 SA 方法中另一個重要的分析工具。資料流程圖對資訊處理邏輯模型的描述，具有直觀、全面、容易理解的優點，但沒有準確、完整地定義圖中各元素。SA 方法要求對於資料流程圖中的所有資料流程、檔和底層加工進行準確、完整定義。這些圖元定義條款彙集在一起即組成資料字典。

在資料字典中給出嚴格的資料定義可以減少分析人員和用戶之間的通信，消除誤解，同時它也是以後進行系統設計及維護的重要依據，是進行系統設計的最有價值的文檔之一。資料流程圖和資料字典應該配合使用，資料流程圖加數據詞典構成系統需求分析規格說明的主要部分。





3.3.4.1 資料字典編寫的基本要求

- (1) 對資料流程圖上的各種元素的定義必須明確、一致且易理解。
- (2) 命名、編號應與資料流程圖一致。
- (3) 對資料流程圖的成分定義與說明無遺漏，無同名異義，或異名同義。
- (4) 格式規範，文字精練，符號正確。

3.3.4.2 資料字典的內容和格式

資料詞典中包括四類條目：資料流程、資料項目、文件、基本加工。

- (1) 資料流程條目：資料流程條目定義一個資料流程的資料項目組成。包括名字、別名、編號、來源去向、包含的資料結構名以及處理特點（如使用頻率、資料量等）和其他注釋資訊（如格式、位置等）。
- (2) 資料項目條目：資料項目是組成資料流程的組成要素，分為基本資料項目和結構型資料項目。基本資料項目是資料處理中基本的不可分割的邏輯單位，如整數、小數、字串、日期、邏輯值等。在資料詞典中通常要求定義基本資料項目的邏輯或者物理格式。結構型資料項目由若干資料項目組成。
- (3) 檔存儲條目：說明存儲檔的名稱、編號、檔組織方式、記錄數及存儲介質等。
- (4) 加工說明條目：說明加工的名稱、編號、輸入/輸出資料流程、加工邏輯概括描述（描述方法見 3.3.5 加工邏輯說明）。

為對上述條目進行準確、清晰、無二義性地描述，應對描述的格式進行規範。表 3-1 給出在資料字典的定義式中常用的符號及含義。

3.3.4.3 資料字典的構造及使用

對資料字典中的條目進行定義時，仍體現自頂向下，逐步細化的思想，直至給出基本資料項目定義為止。如下例：教學管理系統中的課程管理子系統中的資料檔案“學生選課登記表”的格式如表

3-2 所示，在資料字典中的定義格式為：

學生選課登記表= 1{學號+姓名+課程+專業+班級+（備註）}23  
 學號= “00001” .. “99999” 姓名=4{字元}8  
 課程=課程名+課程類型+學分  
 專業= “01” .. “99” ，注：專業代碼，兩位元  
 數字開課時間=年+月+日備註=0{字元}40  
 課程名= “001” .. “999” ，注：課程代碼，三位元數位課程類型  
 =[B | X | R]，注：課程類別一為識別字  
 B= “必選” X= “限選” R= “任選” 學分= “x” 表 3-1 資料字典  
 的定義式中常用的符號及含義

| 符號 | 含義 | 舉例 |
|----|----|----|
|----|----|----|



|            |        |                        |
|------------|--------|------------------------|
| =          | 被定義為   | 日期=年+月+日               |
| +          | 與      | 學生輸入資料=學號+姓名           |
| [...][...] | 或      | 學生輸入資料=[學號 姓名]         |
| {...}      | 重複     | 帳目={帳號+戶名+款額+存期+位址}    |
| m{...}n    | 重複     | 密碼=1{字母+數學}8           |
| (...)      | 選擇     | 登錄資訊=用戶名+(密碼)          |
| “...”      | 基本資料元素 | 印密= “0”                |
| ..         | 連接子    | 學號= “00001” .. “99999” |

表 3-2 學生選課登記表

| 學號 | 姓名 | 課程 | 專業 | 開課時間 | 備註 |
|----|----|----|----|------|----|
|    |    |    |    |      |    |
|    |    |    |    |      |    |
|    |    |    |    |      |    |
|    |    |    |    |      |    |

資料字典是為分析人員及其他開發人員提供了資料流程圖中各種元素的詳細定義，是需求評審以及後續進行系統設計及維護的重要依據。為了便於使用，也需像普通字典一樣，將所定義的所有條目按照一定的編目方法排列起來，並建立索引目錄。可以和日常查詞典一樣，借助於資料詞典就可以查出某個名字的具體含義。

### 3.3.5 加工邏輯說明

在資料流程圖中，只簡單地對每一個加工框進行了編號和命名，沒有表達加工的全部內容。為了理解這些基本加工，像資料流程條目和檔條目一樣，要為每個加工編寫詳細的加工邏輯說明，這些說明可以按照加工的編號次序排列，也可按其他確定的次序排列，形成詞典的加工條目，與資料流程條目和檔條目組成一本完整的資料詞典。在將來的設計中，加工邏輯說明就是對應處理常式設計的基礎。

一般在資料詞典中只列出基本加工的邏輯說明，這是因為隨著自頂向下逐層細化，將一個複雜的系統分解成了許多足夠簡單的基本加工。只要寫出每一個基本加工的全部詳細邏輯功能，再自底向上綜合，就能完成全部邏輯加工。



對資料流程圖的每一個基本加工的邏輯說明，集中描述一個加工“做什麼”，即加工邏輯。加工邏輯是指使用者對這個加工的邏輯要求，即這個加工的輸出資料流程和輸入資料流程的邏輯關係，描述基本加工是如何把輸入資料流程變換為輸出資料流程的加工規則，也包括其他一些與加工有關的資訊，如執行條件、優先順序、執行頻率、出錯處理等。注意加工邏輯說明描述的實現加工的策略，而不是實現加工的細節，即不描述具體的加工過程。

加工邏輯說明中包含的資訊應是充足的、完備的、有用的、準確、易懂的。目前用於描述加工邏輯說明的工具有結構化英語、判定表和決策樹。

### 3.3.5.1 結構化英語 ( Structured English )

結構化英語也叫做問題描述語言 ( Problem Describe Language, PDL )，它是在自然語言基礎上加了一些限制而得到的一種介於自然語言和形式化語言之間的半形式化語言。它使用有限的詞彙和有限的語句來描述加工邏輯。

結構化英語的詞彙表由英語命令動詞、資料詞典中定義的名字、有限的自訂詞和邏輯關係詞 IF\_THEN\_ELSE、WHILE\_DO、REPEAT\_UNTIL、CASE\_OF 等組成。其動詞的含義要具體，不要用抽象的動詞。盡可能少用或不用形容詞和副詞。

結構化自然語言的語法通常分為內外兩層。外層語法描述操作的控制結構，如順序、選擇、迴圈等，這些控制結構將加工中的各個操作連接起來。內層的語法一般沒有限制。語言的正文用基本控制結構進行分割，加工中的操作用自然語言短語表示。其基本控制結構有三種：

- (1) 簡單陳述句結構：避免複合陳述式。
- (2) 判定結構：IF\_THEN\_ELSE 或 CASE\_OF 結構。
- (3) 重複結構：WHILE\_DO 或 REPEAT\_UNTIL 結構。

下面是用結構化語言描述的教學管理系統中，決定學生升留級的加工邏輯說明。

BEGIN

輸入學生成績記錄

```

REPEAT IF 總分 ≥ 600 分 THEN IF 單科成績有不及格
    THEN 發升級通知書 AND 發重修單科通知書
    ELSE 發升級通知書
ELSE IF 單科成績有滿分
    THEN 發留級通知書 AND 發免修單科通知書
    ELSE 發留級通知書
UNTIL 學生成績記錄處理結束
    
```

END

### 3.3.5.2 決策樹 ( Decision Tree )

在某些資料處理問題中，其資料流程圖的處理需要依賴於多個邏輯條件的取值，這些取值構成多種不同的情況，滿足不同條件，執行相應的不同動作。這類問題適合使用決策樹或判定表作為描述加工小說明的工具。



決策樹是一種呈樹狀的圖形工具，適合於描述處理中具有多種策略，要根據若干條件的判定，確定所採用策略的情況。

用決策樹描述的決定學生升留級的加工邏輯說明如圖 3-15 所示。

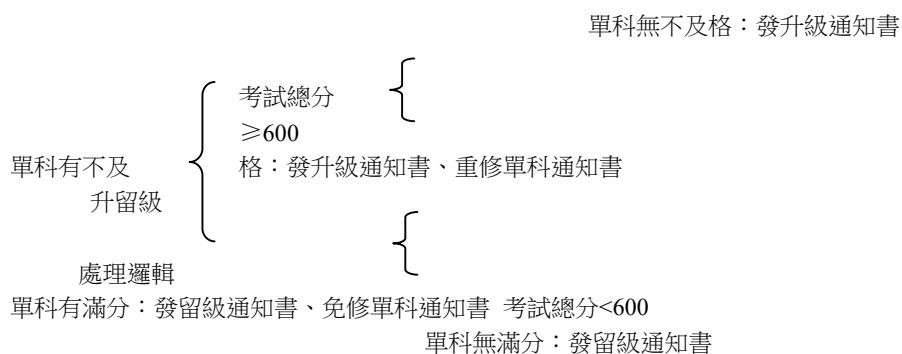


圖 3-15 決定學生升留級決策樹

決策樹具有清晰、直觀、易於使用的優點。但當條件多，而且互相組合時，不容易清楚地表達判斷過程，因此引入判定表。

### 3.3.5.3 判定表 ( Decision Table )

判定表比較適合用來描述資料流程圖中的加工需要依賴於多個邏輯條件的取值，即完成這一加工的一組動作，是由於某一組條件取值的組合而引發的處理情況。判定表的結構如圖 3-16 所示，通常由四部分組成，其中：

- (1) 基本條件區：列出所有可能的基本判斷條件項，通常與次序無關。
- (2) 基本動作區：列出所有可能採取的動作項，通常與次序無關。
- (3) 條件組合區：各種條件給出的多種取值，即多個條件所取真假值的組合。
- (4) 執行動作區：指出在各種條件的特定取值下應採取的動作。在各動作行與條件組合列的交叉處表示在指定條件組合下發生的動作。



图 3-16 判定表的结构

通常將任一條件取值組合及其相應要執行的動作稱為規則（在判定表中貫穿條件項和動作項一列）。顯然，判定表中列出了多少個條件取值的組合，也就有多少條規則。

上例中升留級處理使用判定表描述如圖 3-17 所示。



在實際使用判定表時，常常需要先將其化簡。方法是：如果表中有兩條或更多的規則具有相同的動作，並且其條件項之間存在著某種關係，就可設法將它們合併。例如圖 3-16 中條件項中的第二條件的取值不同，但規則 1 和規則 3 的動作項是一致的，這表明在第一、第三個條件分別取真值和假值時，不論第二條件取何值，都執行同一動作。這樣，可將這兩條規則合併，合併後的第二條件取值用“—”表示，以示與取值無關。類似地，若無關條件項“—”在邏輯上又可包含其他的條件項取值，具有相同動作的規則還可進一步合併。

|          | 規則 1 | 規則 2 | 規則 3 | 規則 4 | 規則 5 | 規則 6 | 規則 7 | 規則 8 |
|----------|------|------|------|------|------|------|------|------|
| 考試總分     | ≥600 | ≥600 | ≥600 | ≥600 | <600 | <600 | <600 | <600 |
| 單科滿分     | 有    | 有    | 無    | 無    | 有    | 有    | 無    | 無    |
| 單科不及格    | 有    | 無    | 有    | 無    | 有    | 無    | 有    | 無    |
| 發升級通知書   | Y    | Y    | Y    | Y    | N    | N    | N    | N    |
| 發單科免修通知書 | N    | N    | N    | N    | Y    | Y    | N    | N    |
| 發留級通知書   | N    | N    | N    | N    | Y    | Y    | Y    | Y    |
| 發單科重修通知書 | Y    | N    | Y    | N    | N    | N    | N    | N    |

圖 3-17 升留級處理邏輯的判定表按此方法對圖

3-17 化簡後的判定表如圖 3-18 所示。

| 考試總分     |   |   |   |   |
|----------|---|---|---|---|
| 單科滿分     | — | — | 有 | 無 |
| 單科不及格    | 有 | 無 | — | — |
| 發升級通知書   | Y | Y | N | N |
| 發單科免修通知書 | N | N | Y | N |
| 發留級通知書   | N | N | Y | Y |
| 發單科重修通知書 | Y | N | N | N |

圖 3-18 化簡後的升留級處理邏輯的判定表

對於需要描述的是由一組操作組成的，其中有些操作是否執行又取決於一組條件，用語言說明難以準確、清楚表達的加工，使用判定表能夠把在什麼條件下、系統應完成哪些動作表達得十分清楚，準確，一目了然，不容易發生錯誤和遺漏。但其缺點是描述迴圈比較困難。有時，判定表可以和結構化英語結合起來使用。



## 3.4 原型法

目前有代表性的軟體需求分析方法除了面向資料流程的結構化分析方法外，還有面向資料結構的 Jackson 方法、物件導向的方法和原型法等。由於原型法（Prototyping）改變了傳統的系統的分析、設計和實現三個順序階段的觀點及自頂向下的開發模式，降低了軟體需求的風險，因此該軟體發展過程模型可作為獲取需求技術並得到廣泛的應用。

### 3.4.1 原型在需求分析中的作用

#### 3.4.1.1 原型的概念

通常，原型是指類比某種產品的原始模型。在軟體發展中，原型（prototype）則是軟體的一個早期可運行的版本，它反映最終系統的部分重要功能和特性。是將系統主要功能和介面通過快速開發，製作為“軟體樣機”，以視覺化的形式展現給用戶，以徵求用戶意見，從而明確無誤地確定用戶需求。同時，原型也可用於徵求內部意見，作為分析和設計的介面之一，便於溝通。

#### 3.4.1.2 原型的作用和特點

如前所述，需求分析的難點是構造一個完整準確的規格說明。特別是對於一些大型的軟體專案，在開發的早期，當系統分析員與使用者一同確定需求時，使用者往往對系統只有一個模糊的想法，不能確定真正需要什麼，很難完全準確地表達對系統的全面要求。尤其是隨著開發工作向前推進，需求還常常會發生變化，同時，對系統運行的效果，只能通過對它的邏輯上的推斷，達到各方對系統的共同理解，而不是在實際運行中判斷評價。所以儘管採取多種方法，仍很難保證所構造的規格說明能將系統的各個方面都描述的完整、準確、一致，並與實際環境相符。

而原型法可在獲得一組基本需求說明後，通過快速分析構造出一個可運行的、滿足使用者基本要求的軟體系統框架。通過運行原型系統，使得使用者通過親身感受得到啟發，提出具體的要求和評價。然後開發者根據使用者的意見對原型加以改進。隨著不斷運行原型、評價和修改，獲得新的原型版本。如此周而復始，逐步澄清模糊和誤解，明確各種需求細節，適應需求的變更，從而提高了最終產品的品質。所以原型法是最準確、最有效的一種需求分析技術。

原型應該具備的第一個特性是“快速”。快速原型的目的是儘快向使用者提供一個可在電腦上運行的目標系統的模型，以便使使用者和開發者盡可能快地達成對目標系統應該“做什麼”的共識。所以，構造原型所做的需求往往是不足的，此時的原型會有某些缺陷，但只要這些缺陷對原型功能的損害不嚴重，不會使使用者對產品的行為產生誤解，就可以忽略。

原型應該具備的第二個特性是“易於修改”。如果原型的第一版不是使用者所需要的，就必須根據使用者的意見迅速地修改它，構建出原型的第二版，以更好地滿足用戶需求。在實際開發軟體產品

時，原型的“修改－試用－回饋”過程可能重複多遍，如果修改耗時過多，勢必延誤軟體發展時間。





### 3.4.1.3 軟體原型的分類

雖然軟體原型化方法是在研究分析階段的方法和技術中產生的，但它更針對傳統方法所面臨的困難。因此，原型化方法也面向軟體發展的其他階段。根據軟體專案的特點和運行原型的不同目的，原型有兩種不同的類型。

(1) 廢棄 (throwaway) 型：先構造一個功能簡單而且品質要求不高的模型系統，針對這個模型系統反復進行分析修改，形成比較好的設計思想，據此設計出更加完整、準確、一致、可靠的最終系統。系統構造完成後，原來的模型系統就被廢棄。

(2) 追加 (addon) 型：先構造一個功能簡單而且品質要求不高的模型系統，作為最終系統的核心，然後通過不斷地擴充修改，逐步追加新要求，最後發展成為最終系統。

採用什麼形式主要取決於軟體專案的特點和開發者的素質，以及支援原型開發的工具和技術，應根據實際情況的特點加以決策。

### 3.4.1.4 構建原型的方法及工具

原型系統不同於最終系統，它需要快速實現，投入運行。因此，必須注意功能和性能上的取捨。在忽略一切暫時不必關心的部分，快速實現原型時，要能充分地體現原型的作用，滿足評價原型的需求。要根據構造原型的目的，明確規定對原型進行考核和評價的內容，如介面形式、系統結構、功能或類比性能等。構造出來的原型可能是一個忽略了某些細節或功能的整體系統結構，也可以僅僅是一個局部，如使用者介面、部分功能演算法程式或資料庫模式等。總之，在使用原型化方法進行軟

件開發之前，必須明確使用原型的目的，從而決定分析與構造內容的取捨。對原型的基本要求包括：

- (1) 體現主要的功能。
- (2) 提供基本的介面風格。
- (3) 展示比較模糊的部分，以便於確認或進一步明確，防患於未然。
- (4) 原型最好是可運行的，至少在各主要功能模組之間能夠建立相互連接。

為了快速地構建和修改原型，通常使用下述 3 種方法和工具。

#### 1. 第四代技術

第四代技術包括眾多資料庫查詢和報表語言、程式和應用系統生成器以及其他非常高級的非過程語言。第四代技術使得軟體工程師能夠快速地生成可執行的代碼，因此，它們是較理想的快速原型工具。

#### 2. 可重用的軟體構件

另外一種快速構建原型的方法，是使用一組已有的軟體構件（也稱為元件）來裝配（而不是從頭構造）原型。軟體構件可以是資料結構（或資料庫），或軟體體系結構構件（即程式），或過程構件（即模組）。必須把軟體構件設計成能在不知其內部工作細節的條件下重用。應該注意，現有的軟體也可以被用作“新的或改進的”產品的原型。



### 3 · 形式化規格說明和原型環境

在過去的 20 多年中，人們已經研究出許多形式化規格說明語言和工具，用於替代自然語言規格說明技術。今天，形式化語言的宣導者正在開發互動式環境，以便可以調用自動工具把基於形式語言的規格說明翻譯成可執行的程式碼，用戶能夠使用可執行的原型代碼去進一步精化形式化的規格說明。

#### 3.4.2 快速原型開發過程

原型的開發和使用過程叫做原型生存期。圖 3-19 給出了原型法開發過程，下面分別對過程中的關鍵步驟進行討論。

##### 1 · 快速分析

在分析者和用戶的緊密配合下，快速確定軟體系統的基本要求。根據原型所要體現的特性（或介面形式，或處理功能，或總體結構，或類比性能等），描述基本規格說明，以滿足開發原型的需要。快速分析的關鍵是要注意選取分析和描述的內容，圍繞使用原型的目標，集中力量，確定局部的需求說明，從而儘快開始構造原型。

##### 2 · 構造原型

在快速分析的基礎上，根據基本規格說明，儘快實現一個可運行的系統。為此需要強有力的軟體工具的支援，並忽略最終系統在某些細節上的要求，如安全性、健壯性、異常處理等。主要考慮原型系統應充分反映的待評價的特性。

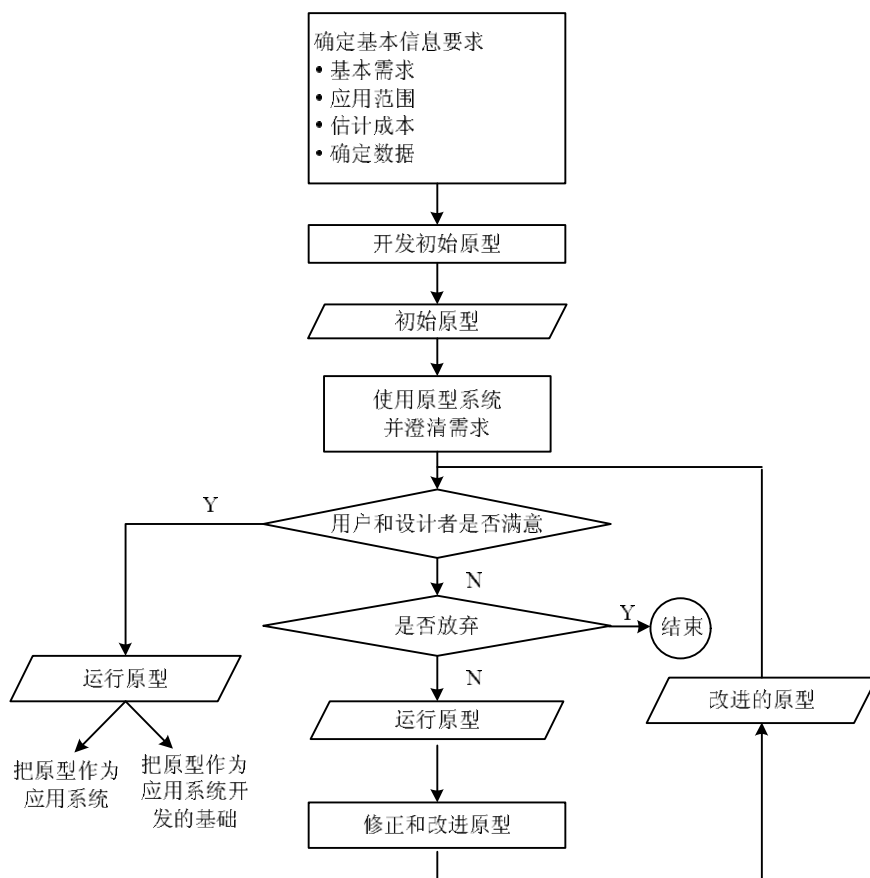


图 3-19 原型法开发过程

提交一個初始原型所需要的時間，根據問題的規模、複雜性、完整程度的不同而不同。一般在 3~6 周提交一個系統的初始原型，最大限度不能超過兩個月，兩個月後提交的應是一個系統而不是一個原型。

### 3. 運行和評價原型

這是頻繁通信、發現問題、消除誤解的重要階段。由於原型忽略了一些內容，它集中反映了要評價的特性，外觀看起來可能會有些殘缺不全。用戶要在開發者的指導下試用原型，在試用的過程中考核評價原型的特性，分析其運行結果是否滿足規格說明的要求，以及規格說明的描述是否滿足用戶的願望。糾正誤解和錯誤，增補新的要求，並為滿足因環境變化或用戶的新設想而引起系統需求的變動並提出全面的修改意見。

### 4. 修正和改進

根據修改意見進行修改。大多數原型不合適的部分可以修正，使之成為新模型的基礎。如果是由於規格說明不準確（有多義性或者未反映用戶要求）、不完整（有遺漏）、不一致，或者需求有所變動或增加，則首先要修改並確定規格說明，然後再重新構造或修改原型。但若出現因為嚴重的理解錯誤而使正常操作的原型與用戶要求相違背時，有可能會產生廢品，應當立即放棄，而不能再湊



合。如果用修改原型的過程代替快速分析，就形成了原型開發的反覆運算過程。開發者和用戶在一次次的反覆運算過程中不斷將原型完善，以接近系統的最終要求。

在修改原型的過程中會產生各種各樣的積極的或消極的影響，為了控制這些影響，應當有一個詞典，用以定義應用的資訊流以及各個系統成分之間的關係。另外，在使用者積極參與的情況下，保留改進前後的兩個原型，一旦使用者需要時可以退回，而且貫穿地演示兩個可供選擇的物件，有助於決策。

#### 5. 判定原型完成

經過修改或改進原型，達到參與者一致認可，則原型開發的反覆運算過程可以結束。為此，應判斷有關應用的實質是否已經掌握，反覆運算週期是否可以結束等。判定的結果有兩個不同的轉向，一是繼續反覆運算驗證，一是進行詳細說明。

#### 6. 判斷原型細部是否說明

判斷組成原型的細部是否需要嚴格地加以說明。原型化方法允許對系統必要成分進行嚴格地詳細地說明，例如將需求轉化為報表，給出統計數字等。

#### 7. 原型細部的說明

必要時，對那些不能通過模型進行說明的成分，如系統的輸入、輸出、加工，系統的邏輯功能、資料庫組織、系統的可靠性、使用者地位等，通過檔加以說明，並利用螢幕進行討論和確定。嚴格說明的成分要作為原型化方法的模型編寫詞典，以得到一個統一的連貫的規格說明提供給開發過程。

#### 8. 判定原型效果

考察使用者新加入的需求資訊和細部說明資訊，看其對模型效果產生的影響，判斷是否會影響模組的有效性，如果模型效果受到影響，甚至導致模型失效，則要進行修正和改進。

#### 9. 整理原型和提供文檔

整理原型的目的是為進一步開發提供依據。原型的初期需求模型是一個自動的文檔。

總之，利用原型化技術，可為軟體的開發提供一種完整的、靈活的、近似動態的規格說明方法。

### 3.5 小結

需求分析階段的任務就是實現需求工程，包括需求開發和需求管理兩部分。本章介紹了需求分析的發現、求精、建模、規格說明和複審的過程，及需求管理的內容；討論了獲取需求的方法及需求分析的原則。

傳統軟體工程方法學使用結構化分析技術，完成需求分析的工作。結構化分析方法從可行性研究階段得到的資料流程圖出發，在用戶的協助下，面向資料流程自頂向下逐步求精，最終構造出系統的邏輯模型，該模型包括一套分層資料流程圖，並附加資料字典和加工邏輯說明。資料字典描述在資料流程圖中出現的所有資料物件及控制資訊的特性，並給出它們的準確定義。



原型法改變了系統的分析、設計和實現三個順序階段的觀點，改變了傳統的自頂向下的開發模式，降低了軟體需求的風險，因此得到了廣泛的應用。實踐表明，快速建立軟體原型是最準確、最有效和最強大的需求分析技術。快速原型應該具備的基本特性是“快速”和“容易修改”，因此，必須用適當的軟體工具支援快速原型技術。通常使用第四代技術、可重用的軟體構件、形式化規格



說明與原型環境，快速地構建和修改原型。

在需求分析階段還應該寫出軟體需求規格說明書，經過嚴格評審並得到用戶確認之後，作為這個階段的最終成果。通常主要從一致性、完整性、現實性和有效性等 4 個方面複審軟體需求規格說明書。

### 習題 3

1. 為什麼可行性研究代替不了需求分析？
2. 需求分析階段的出發點和最終結果各是什麼？
3. 為什麼要進行需求分析？通常對軟體系統有哪些需求？
4. 怎樣與用戶有效地溝通以獲取用戶的真實需求？
5. 資料流程圖和程式流程圖是哪些階段採用的圖形工具？它們有何不同？

6. 某儲蓄所的存、取款業務需求是：儲戶將填好的存（取）款單及存摺交業務員，業務員進行分類處理。如存摺不符合規定或填寫有誤，將存摺及存（取）款單返還儲戶；如果是存款，將存摺及存款單、現金交存款業務員處理，存款業務員取出底賬登記後，將存摺還給儲戶；如果是取款，將存摺及取款單交取款業務員處理，取款業務員取出底賬登記後，將存摺與現金交付儲戶。試畫出該系統的頂層和 0 層資料流程圖。

7. 畫出習題 2 中第 4、第 5 題的資料流程圖。

8. 某考務中心準備開發一個“自學考試考務管理系統”，經調研該系統功能如下：

- (1) 對考生填寫的報名單進行審查，對合格的新生，編好准考證發給他，匯總後的報名單送給閱卷站。
- (2) 給合格的考生製作考試通知單，將考試科目、時間、地點安排告訴考生。
- (3) 對閱卷站送來的成績進行登記，按當年標準審查單科合格者，併發成績單，對所考專業各科成績均合格者，發放大專畢業證。
- (4) 對成績進行分類統計，產生統計報表。(5) 考生可按准考證號隨時查詢各科成績。

請按結構化分析方法進行分析，並畫出 DFD（細化至三層），構造資料字典。

9. 簡述原型法的特點。

10. 簡述結構化分析方法的步驟。

11. 郵寄包裹收費標準如下：

若收件地點在 1000 千米以內，普通件每千克 2 元，掛號件每千克 3 元。若收件地點在 1000 千米以外，普通件每千克 2.5 元，掛號件每千克 3.5 元；若重量大於 30 千克，超出部分每千克加收

0.5 元。請繪製確定收費的決策樹和判定表。